

Java Arrays & Strings

Complete Notes with Diagrams & Full Explanations

Covers declaration, memory model, multi-dimensional arrays, the Arrays utility class, String immutability, the String Pool, StringBuilder, and the most common interview algorithms — with diagrams for every core concept.

Prepared by Akshit Sharma



8510968871



akshubhardwaj231@gmail.com



akshbuilds.tech

Part 1 — Arrays

1.1 What is an Array?

An array is a collection of elements of the same data type, stored in contiguous memory locations, and accessed through a common name using an index.

- Arrays in Java are objects (stored on the heap), even when they hold primitive types.
- Array indices start at 0 and go up to length - 1.
- Once created, the size of an array is fixed — it cannot grow or shrink.

1.2 Array Memory Model (Diagram)

An array variable (like `arr`) is a reference stored on the Stack. It points to the actual block of elements, which live together in one contiguous block on the Heap. This is why array access by index is very fast ($O(1)$) — the JVM just computes an offset from the start address.

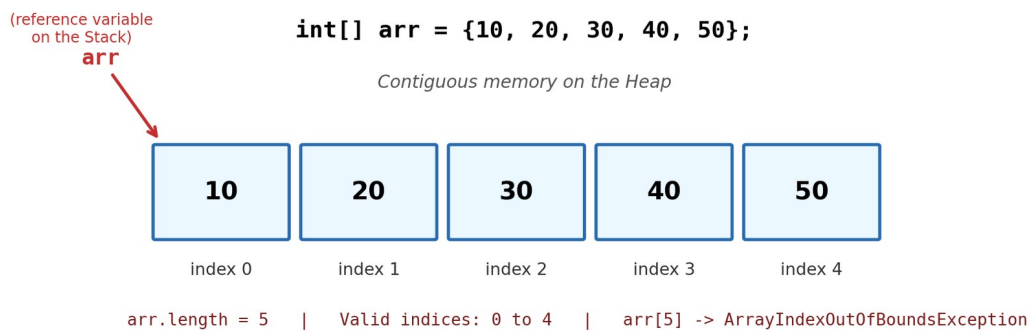


Figure 1: An `int[]` of 5 elements — the reference on the stack points to contiguous heap memory.

1.3 Declaring and Creating Arrays

```
// Declaration
int[] arr;           // preferred style
int arr2[];          // C-style, also valid

// Creation (memory allocation)
arr = new int[5];    // array of 5 ints, default value 0

// Declaration + Creation
int[] arr = new int[5];

// Declaration + Initialization
int[] arr = {10, 20, 30, 40, 50};
int[] arr2 = new int[]{10, 20, 30};
```

Default values when an array is created with `new`:

Type	Default Value
int / long / short / byte	0
double / float	0.0

Type	Default Value
boolean	false
char	'\u0000'
Object / String	null

1.4 Accessing & Modifying Array Elements

```
int[] arr = {10, 20, 30};
System.out.println(arr[0]);    // 10
arr[1] = 99;                  // modify element
System.out.println(arr.length); // 3 (property, not method)
```

⚠ Accessing an invalid index throws `ArrayIndexOutOfBoundsException`.

1.5 Looping Through Arrays

```
int[] arr = {1, 2, 3, 4, 5};

// Standard for loop
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}

// Enhanced for loop (for-each) – use when index is not needed
for (int val : arr) {
    System.out.println(val);
}
```

1.6 Multi-Dimensional Arrays

Java doesn't have 'true' 2D arrays — a 2D array is really an array of arrays. The outer array stores references to inner (row) arrays, which can even have different lengths (a 'jagged' array).

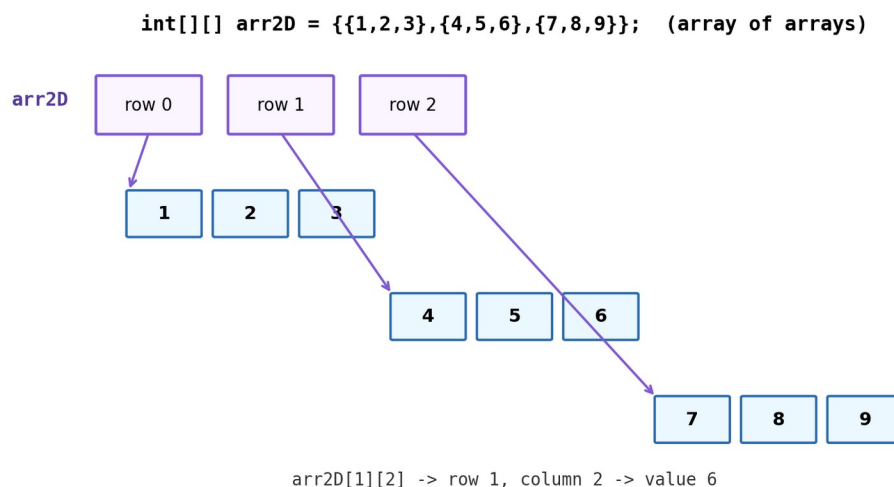


Figure 2: A 2D array is an outer array of row-references, each pointing to its own row array.

```
// 2D array (matrix)
int[][] matrix = new int[3][4]; // 3 rows, 4 columns
```

```

int[][] arr2D = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Accessing
System.out.println(arr2D[1][2]); // 6 (row 1, col 2)

// Traversing a 2D array
for (int i = 0; i < arr2D.length; i++) {
    for (int j = 0; j < arr2D[i].length; j++) {
        System.out.print(arr2D[i][j] + " ");
    }
    System.out.println();
}

// Jagged array (rows of different lengths)
int[][] jagged = new int[3][];
jagged[0] = new int[]{1};
jagged[1] = new int[]{1, 2};
jagged[2] = new int[]{1, 2, 3};

```

1.7 Array of Objects / Strings

```

String[] names = new String[3];
names[0] = "Alice";
names[1] = "Bob";
// names[2] is null (not set)

String[] fruits = {"Apple", "Banana", "Cherry"};

```

1.8 Passing Arrays to Methods

Arrays are passed by reference (the reference/address is copied, not the data). So changes made inside a method affect the original array.

```

static void doubleValues(int[] arr) {
    for (int i = 0; i < arr.length; i++) arr[i] *= 2;
}

public static void main(String[] args) {
    int[] nums = {1, 2, 3};
    doubleValues(nums);
    System.out.println(Arrays.toString(nums)); // [2, 4, 6]
}

```

1.9 The java.util.Arrays Utility Class

Import: `import java.util.Arrays;`

Method	Purpose
<code>Arrays.toString(arr)</code>	Print array as string

Method	Purpose
<code>Arrays.sort(arr)</code>	Sort array (ascending, in-place)
<code>Arrays.sort(arr, from, to)</code>	Sort a portion of the array
<code>Arrays.fill(arr, val)</code>	Fill all elements with a value
<code>Arrays.equals(arr1, arr2)</code>	Compare two arrays element-by-element
<code>Arrays.copyOf(arr, newLength)</code>	Copy into a new array (resize)
<code>Arrays.copyOfRange(arr, from, to)</code>	Copy a range
<code>Arrays.binarySearch(arr, key)</code>	Search in a sorted array
<code>Arrays.asList(arr)</code>	Convert array to a List (fixed-size)
<code>Arrays.deepToString(arr2D)</code>	Print 2D / nested arrays properly

```
import java.util.Arrays;

int[] arr = {5, 2, 8, 1, 9};
Arrays.sort(arr);
System.out.println(Arrays.toString(arr)); // [1, 2, 5, 8, 9]

int idx = Arrays.binarySearch(arr, 8);    // 3

int[] copy = Arrays.copyOf(arr, 7);        // extra slots filled with 0
int[] range = Arrays.copyOfRange(arr, 1, 4); // elements at index 1,2,3

int[][] m = {{1,2},{3,4}};
System.out.println(Arrays.deepToString(m)); // [[1, 2], [3, 4]]
```

1.10 Sorting an Array in Descending Order

Primitive arrays need boxed types (`Integer[]`) with a comparator, since `Arrays.sort` on `int[]` only sorts ascending.

```
Integer[] arr = {5, 2, 8, 1, 9};
Arrays.sort(arr, Collections.reverseOrder());
```

Or manually reverse the array after an ascending sort.

1.11 Common Array Algorithms (Interview Favorites)

Find max / min

```
int max = arr[0];
for (int v : arr) if (v > max) max = v;
```

Sum & Average

```
int sum = 0;
for (int v : arr) sum += v;
double avg = (double) sum / arr.length;
```

Reverse an array (in-place, two-pointer)

```
int left = 0, right = arr.length - 1;
```

```
while (left < right) {
    int temp = arr[left];
    arr[left] = arr[right];
    arr[right] = temp;
    left++; right--;
}
```

Linear Search — $O(n)$

```
int target = 5, index = -1;
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == target) { index = i; break; }
}
```

Binary Search — $O(\log n)$ (array must be sorted)

```
int binarySearch(int[] arr, int target) {
    int lo = 0, hi = arr.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Remove duplicates (using a Set)

```
Set<Integer> set = new LinkedHashSet<>();
for (int v : arr) set.add(v);
```

Check if array is sorted

```
boolean sorted = true;
for (int i = 0; i < arr.length - 1; i++) {
    if (arr[i] > arr[i+1]) { sorted = false; break; }
}
```

1.12 Key Points / Gotchas — Arrays

- `arr.length` is a field, not a method (no parentheses), unlike `String.length()`.
- Arrays are fixed size — use `ArrayList` for dynamic sizing.
- Arrays are objects; the default value of an array reference is `null`.
- `==` on arrays compares references, not contents — use `Arrays.equals()`.
- Multi-dimensional arrays are really arrays of arrays, which allows jagged arrays.

Part 2 — Strings

2.1 What is a String in Java?

A String is a sequence of characters. In Java, String is a class (`java.lang.String`), not a primitive type, and String objects are immutable — once created, the content cannot change.

```
String s1 = "Hello";           // String literal
String s2 = new String("Hello"); // Using constructor
```

2.2 String Immutability

Once a String object is created, its value cannot be changed. Any operation that appears to 'modify' a string actually creates a brand-new String object, leaving the original untouched.

```
String s = "Hello";
s.concat(" World"); // does NOT change s
System.out.println(s); // "Hello" (unchanged)

s = s.concat(" World"); // must reassign to capture new string
```

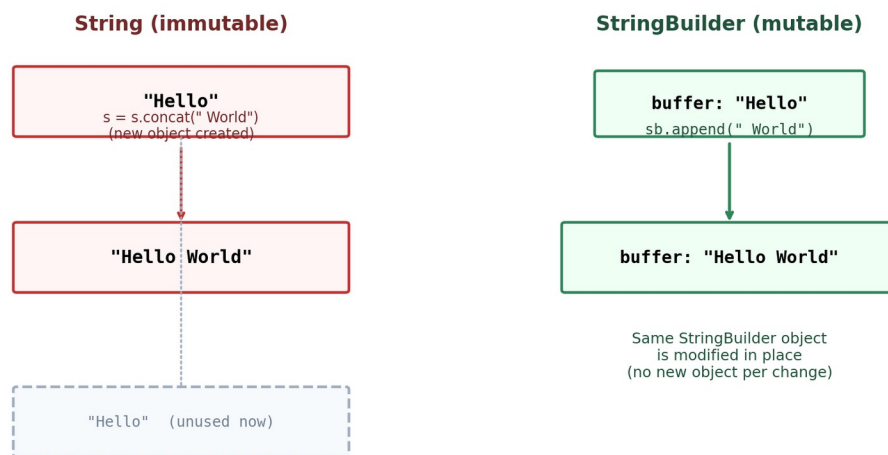


Figure 3: String operations create new objects; StringBuilder mutates the same object in place.

2.3 The String Pool (Diagram)

Java maintains a special memory region called the String Constant Pool inside the heap. String literals are stored here and reused: if two literals have the same content, they point to the same pooled object. Using the `new` keyword bypasses the pool and always creates a fresh object on the heap.

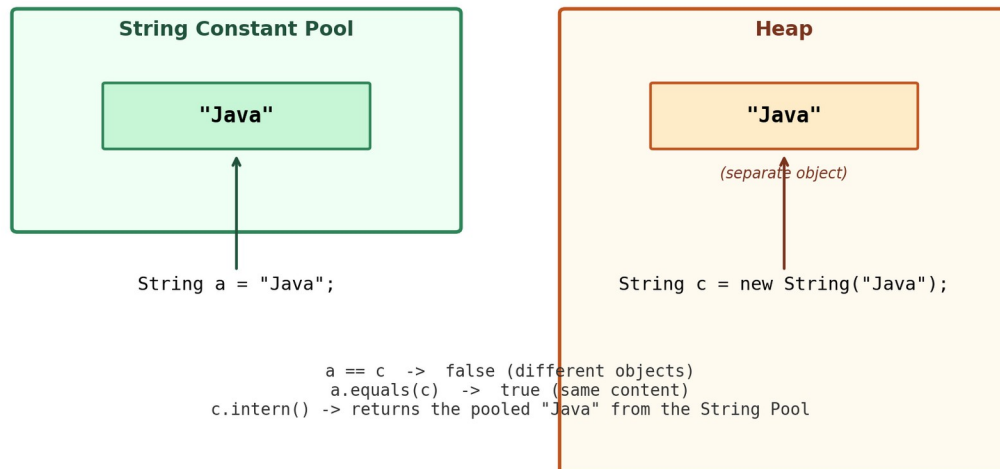


Figure 4: A literal is placed in the String Pool; new String(...) always creates a separate heap object.

```

String a = "Java";
String b = "Java";
System.out.println(a == b);    // true – both point to the same pooled literal

String c = new String("Java");
System.out.println(a == c);    // false – new object created on heap, not pool
System.out.println(a.equals(c)); // true – same content

String d = c.intern();        // forces "Java" from pool
System.out.println(a == d);    // true

```

! Rule of thumb: Always use .equals() to compare string content. Use == only to check if two references point to the same object.

2.4 Creating Strings

```

String s1 = "text";           // literal (pool)
String s2 = new String("text"); // heap object
char[] chars = {'J','a','v','a'};
String s3 = new String(chars); // from char array
String s4 = String.valueOf(123); // from int -> "123"
String s5 = "" + 123;         // concatenation trick

```

2.5 Important String Methods

Method	Description	Example → Result
length()	Number of characters	"Hello".length() → 5
charAt(i)	Character at index i	"Hello".charAt(1) → 'e'
substring(start)	From index to end	"Hello".substring(2) → "llo"
substring(start, end)	start (incl) to end (excl)	"Hello".substring(1,4) → "ello"
indexOf(str)	First index of substring (–1 if absent)	"Hello".indexOf("l") → 2
lastIndexOf(str)	Last index of substring	"Hello".lastIndexOf("l") → 3

Method	Description	Example → Result
contains(str)	Checks if substring exists	"Hello".contains("ell") → true
equals(str)	Case-sensitive content comparison	"abc".equals("ABC") → false
equalsIgnoreCase(str)	Case-insensitive comparison	"abc".equalsIgnoreCase("ABC") → true
compareTo(str)	Lexicographic comparison (returns int)	"a".compareTo("b") → negative
toUpperCase()	Converts to uppercase	"abc".toUpperCase() → "ABC"
toLowerCase()	Converts to lowercase	"ABC".toLowerCase() → "abc"
trim()	Removes leading/trailing whitespace	" hi ".trim() → "hi"
strip()	Like trim, Unicode-aware (Java 11+)	" hi ".strip() → "hi"
replace(old,new)	Replaces all occurrences	"aaa".replace('a','b') → "bbb"
replaceAll(regex,new)	Regex-based replace	"a1b2".replaceAll("\\d","") → "ab"
split(regex)	Splits into a String[]	"a,b,c".split(",") → [a, b, c]
concat(str)	Concatenates strings	"Hi".concat(" there") → "Hi there"
isEmpty()	Checks length == 0	"".isEmpty() → true
isBlank()	Checks empty/whitespace only (Java 11+)	" ".isBlank() → true
toCharArray()	Converts to char[]	"abc".toCharArray()
startsWith(str)	Checks prefix	"Hello".startsWith("He") → true
endsWith(str)	Checks suffix	"Hello".endsWith("lo") → true
format(...)	Formatted string (like printf)	String.format("%d-%s",1,"a")
join(delim, ...)	Joins strings with a delimiter	String.join("-", "a", "b") → "a-b"
repeat(n)	Repeats string n times (Java 11+)	"ab".repeat(2) → "abab"
matches(regex)	Tests against a regex	"123".matches("\\d+") → true

2.6 String Concatenation

```
String s = "Hello" + " " + "World"; // compile-time optimized (uses StringBuilder internally)
```

```
// In a loop, avoid repeated + concatenation – creates many objects
String result = "";
for (int i = 0; i < 5; i++) {
    result += i; // inefficient – new String object each time
}
```

2.7 StringBuilder & StringBuffer (Mutable Strings)

Since String is immutable, repeated modification (e.g. inside loops) is inefficient. Use StringBuilder (not thread-safe, faster) or StringBuffer (thread-safe, synchronized) instead.

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) {
```

```

        sb.append(i);
    }
    System.out.println(sb.toString()); // "01234"

```

Common StringBuilder Methods:

Method	Purpose
append(x)	Add to the end
insert(idx, x)	Insert at position
delete(start,end)	Remove characters in range
deleteCharAt(idx)	Remove one character
reverse()	Reverse the sequence
replace(start,end,str)	Replace range with new string
charAt(idx) / setCharAt(idx,ch)	Get/set a character
length()	Current length
toString()	Convert back to String

```

StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");      // "Hello World"
sb.insert(5, ",");        // "Hello, World"
sb.reverse();             // "dlroW ,olleH"
sb.reverse();             // back to "Hello, World"
sb.deleteCharAt(5);       // "Hello World"

```

String vs StringBuilder vs StringBuffer:

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread-safe	Yes (immutable)	No	Yes
Performance	Slower for many edits	Fast	Slower than StringBuilder (sync overhead)
Use case	Fixed / rare-change text	Single-threaded heavy editing	Multi-threaded heavy editing

2.8 Comparing Strings

```

String a = "apple", b = "Apple";

a.equals(b);           // false (case-sensitive)
a.equalsIgnoreCase(b); // true
a.compareTo(b);        // > 0 (lowercase 'a' > uppercase 'A' in ASCII)
a.compareToIgnoreCase(b); // 0

```

2.9 String and char[] / Arrays

```

String s = "Hello";
char[] arr = s.toCharArray(); // String -> char[]

```

```
String back = new String(arr);    // char[] -> String
String back2 = String.valueOf(arr);

// Splitting a sentence into words
String sentence = "Java is fun";
String[] words = sentence.split(" "); // ["Java","is","fun"]
```

2.10 Common String Algorithms (Interview Favorites)

Reverse a string

```
String reverse(String s) {
    return new StringBuilder(s).reverse().toString();
}
// Manual approach without StringBuilder
String reverseManual(String s) {
    char[] arr = s.toCharArray();
    int left = 0, right = arr.length - 1;
    while (left < right) {
        char t = arr[left]; arr[left] = arr[right]; arr[right] = t;
        left++; right--;
    }
    return new String(arr);
}
```

Check Palindrome

```
boolean isPalindrome(String s) {
    String clean = s.toLowerCase().replaceAll("[^a-z0-9]", "");
    return clean.equals(new StringBuilder(clean).reverse().toString());
}
```

Count vowels / characters

```
int countVowels(String s) {
    int count = 0;
    for (char c : s.toLowerCase().toCharArray()) {
        if ("aeiou".indexOf(c) != -1) count++;
    }
    return count;
}
```

Check if two strings are Anagrams

```
boolean isAnagram(String s1, String s2) {
    if (s1.length() != s2.length()) return false;
    char[] a1 = s1.toCharArray();
    char[] a2 = s2.toCharArray();
    Arrays.sort(a1);
    Arrays.sort(a2);
    return Arrays.equals(a1, a2);
}
```

Count frequency of each character

```
Map<Character, Integer> freq = new HashMap<>();
```

```
for (char c : s.toCharArray()) {  
    freq.put(c, freq.getOrDefault(c, 0) + 1);  
}
```

Find first non-repeating character

```
char firstNonRepeating(String s) {  
    Map<Character, Integer> count = new LinkedHashMap<>();  
    for (char c : s.toCharArray()) count.put(c, count.getOrDefault(c, 0) + 1);  
    for (char c : s.toCharArray()) if (count.get(c) == 1) return c;  
    return '\0'; // not found  
}
```

Remove duplicate characters (preserve order)

```
String removeDuplicates(String s) {  
    StringBuilder sb = new StringBuilder();  
    Set<Character> seen = new LinkedHashSet<>();  
    for (char c : s.toCharArray()) {  
        if (seen.add(c)) sb.append(c);  
    }  
    return sb.toString();  
}
```

2.11 Key Points / Gotchas — Strings

- Strings are immutable — every 'modifying' method returns a new String.
- Use `.equals()` for content comparison, never `==` (except intentional reference checks).
- String literals are pooled; new `String(...)` always creates a new heap object.
- For heavy string manipulation (loops, concatenation), prefer `StringBuilder`.
- String implements `CharSequence`, `Comparable<String>`, and `Serializable`.
- `substring(start, end)`: the end index is exclusive.

Part 3 — Arrays vs Strings: Quick Comparison

Feature	Array	String
Mutability	Mutable (elements can change)	Immutable
Type	Can hold primitives or objects	Holds only characters (internally char[]/byte[])
Size	Fixed after creation	Fixed after creation (new object needed to 'change')
Memory	Heap	Heap (String Pool for literals)
Common use	Storing collections of data	Storing / manipulating text

Quick Reference Cheat Sheet

```
// ARRAYS
int[] arr = {5, 3, 1, 4, 2};
Arrays.sort(arr);
System.out.println(Arrays.toString(arr));
int idx = Arrays.binarySearch(arr, 4);

// STRINGS
String s = " Hello World ";
s = s.trim().toLowerCase();
String[] words = s.split(" ");
StringBuilder sb = new StringBuilder();
for (String w : words) sb.append(w).append("-");
System.out.println(sb.toString());
```

Tip: For coding interviews, master these patterns — two-pointer technique, sliding window, HashMap frequency counting, and StringBuilder usage — since most array/string problems reuse them.

Prepared by Akshit Sharma

 8510968871 |  akshubhardwaj231@gmail.com |  akshbuilds.tech